

Mobile Prepaid Recharge Application – Preparatory Documentation

1. Project Overview

1.1. **Project Name :** PrepaidGo – Mobile Prepaid Recharge Application

1.2. **Purpose and Goals:**

PrepaidGo is a web-based prepaid mobile recharge application for Mobi-Comm Service Ltd. that enables users to recharge their mobile numbers, manage plans, and track transaction history efficiently. The goal is to provide a secure, reliable, and user-friendly platform for seamless prepaid mobile recharges.

1.3. **Expected Outcomes:**

- A responsive and intuitive user interface.
- Role Based Authentication and Authorization
- Easy plan selection through various filter options and recharge tracking.
- Quick recharge feature for guest users.
- Efficient admin management of recharge plans and user activities.

2. Scopes & Objectives

2.1. **Features to be Included:**

- User authentication (JWT-based security, OTP login)
- Plan selection and mobile recharge
- Secure payments and transaction tracking
- Recharge history for registered users
- Quick recharge without login
- Mobile number portability request (port-in feature)
- Admin management of recharge plans and users

2.2. **Features Explicitly Excluded:**

- Postpaid bill payments

- Wallet-based transactions
- Messaging system for customer support
- Real-time plan usage alerts

3. Technical Specification

3.1. Architecture Overview:

Client-side: Frontend UI communicates with backend APIs through fetch API.

Server-side:

- **Controller Layer:** Handles HTTP requests and responses.
- **Service Layer:** Contains business logic and processes requests from the controller.
- **Repository Layer:** Directly interacts with the database using JPA/Hibernate and implements JPA Repository for efficient database interaction.
- **DTO (Data Transfer Object) Layer:** Encapsulates data for transfer between layers , used to prevent direct exposure of entity models and to customize the JSON responses.
- **Security Layer:** Implements JWT-based authentication and authorization.

Backend APIs

S.No	BACKEND APIs	USAGE
1	admin/profile	-to fetch admin details to be displayed on admin profile
2	admin/logout	-used to securely logout admin , by revoking the authorization token
3	admin/update/email	-to update the admin email id
4	admin/update/password	-to update the admin password
5	auth/admin/register	-to securely register the admin by encrypting password using BCryptPasswordEncoder

6	auth/admin/login	-to securely login the admin using basic authentication(username and password)
7	auth/otp/generate	-to generate and send the otp
8	auth/otp/verify	-to verify the otp stored in redis with the user entered otp
9	plans/active	-to fetch all active plans to display it to subscribers
10	admin/plans/all	-to fetch all plans (both active and inactive) to display it to admin
11	admin/category/add	-to add the category (admin only has the authority)
12	admin/plans/add	-to add a plan (admin only has the authority)
13	admin/plans/update	-to update a plan
14	admin/plans/activate/{id}	-to activate a inactive plan
15	admin/plans/delete/{id}	-to deactivate a plan (make the plan status as inactive)
16	plans/{id}	-to fetch plan by id
17	plans/category/{category}	-to fetch plans by particular category
18	plans/categories	-to fetch all the available categories
19	plans/filter	-to fetch the filtered plans
20	plans/recharge/confirm	-to confirm the recharge done by subscribers
21	auth/validate/number	-to validate the number entered by subscriber
22	auth/subscriber/number	-to validate the number and return the details of corresponding subscriber whose number is entered
23	subscriber/profile	-to fetch the details of subscriber
24	Auth/guest/profile	-to fetch the details of subscriber when a guest is making a recharge(since subscriber/profile is authorized subscribers)
25	subscriber/fullname	-to fetch the fullname of the subscriber to display in navbar

26	subscriber/profile/transactions	-to fetch the transactions done by a subscriber to display it in his/her profile
27	subscriber/profile/recharges	-to fetch the recharges done by a subscriber to display it in his/her profile
28	subscriber/transaction	-to fetch a particular transaction detail using transaction number
29	subscriber/active-plan	-to fetch the current active plan of a subscriber
30	subscriber/logout	-to securely logout a subscriber , by revoking the authorization token
31	admin/subscriber/expiring	-to fetch all subscribers whose plan is expiring in next 3 days
32	admin/subscribers	-to fetch all subscribers

The APIs starts with 'admin' are authorized only for Admin

The APIs starts with 'subscribers' are authorized for both admin and subscribers

The APIs starts with plans and auth are not authorized for any roles it is open to all users.

Database: Stores user data, recharge plans, and transaction records.

3.2. Technology Stack:

Frontend:

- Figma - UI/UX Design
- HTML, CSS, JavaScript, Bootstrap
- Fetch API for backend communication
- Local and session storage for user sessions

Backend:

- Spring Boot with Maven
- Spring Security with JWT for authentication

- Redis for OTP storage
- JavaMailSender for email notifications

Database:

- MySQL Database for storing user and transaction details

3.3. Integration Details:

- Redis Database integration for efficient OTP Storage and verifications.
- External SMTP service for email notifications.

4. User Roles & Functionalities

4.1. User Roles:

Subscriber: End-user performing prepaid mobile recharges.

Admin: Manages recharge plans, users, and reports.

4.2. Functionalities:

Subscribers:

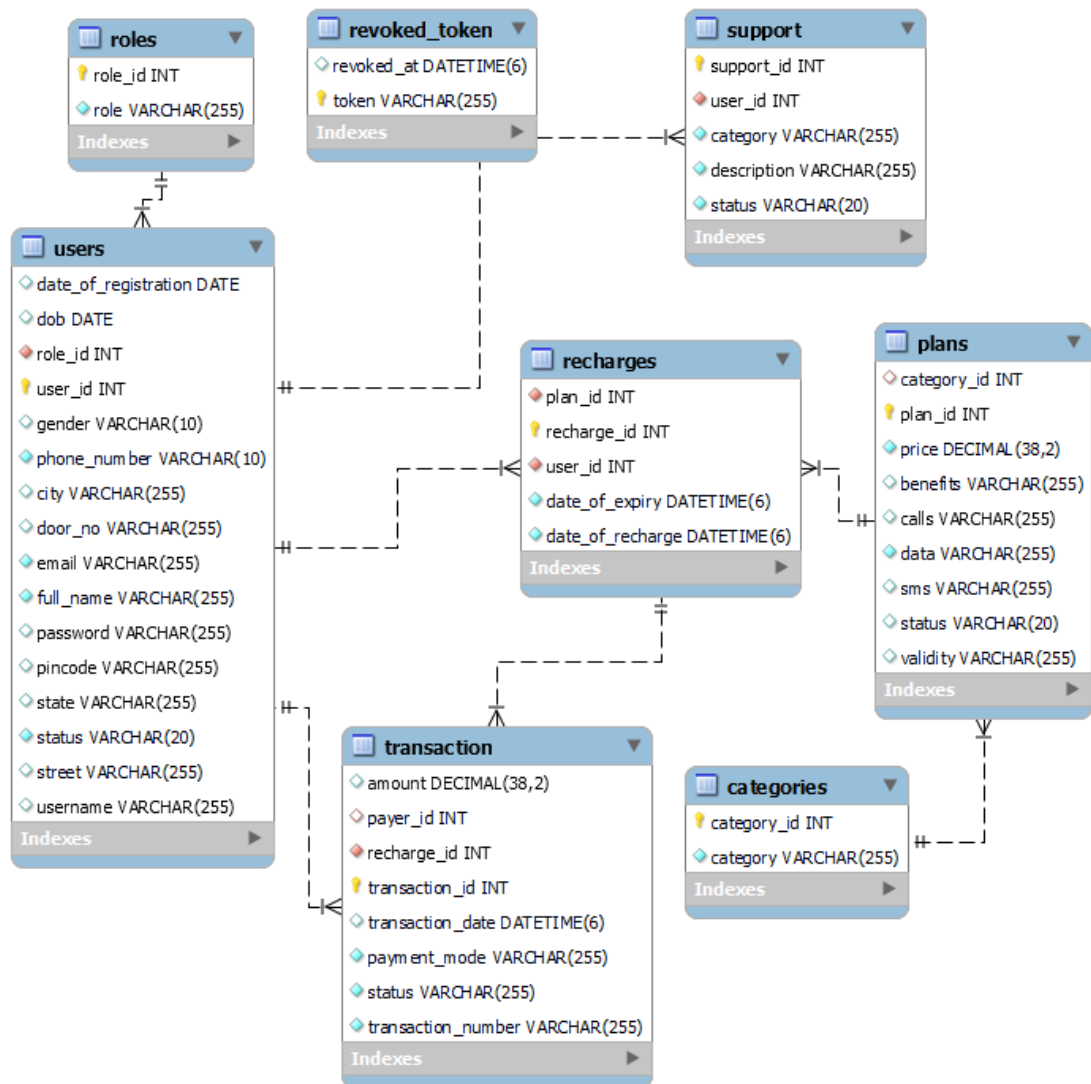
- Register/login via OTP
- View available recharge plans
- Make secure payments
- Track recharge and transaction history
- Use quick recharge feature
- Request mobile number portability (yet to be implemented)

Admin:

- Manage prepaid plans (add, edit, delete, view category-wise)
- View and analyze recharge and transaction reports
- Manage user accounts and authentication settings

5. Data Flow & Structure

5.1. High-Level ERD (Entity-Relationship Diagram):



5.2. Data Model Design:

Tables and Associations:

Entity	Relationship	Entity
Role	One - Many	Users
Users	One - Many	Support
Users	Many - One	Plans
Category	One - Many	Plans
Users	One - Many	Recharges
Recharges	One - One	Transaction
Recharges	Many - One	Plans
Transaction	Many - One	Users

6. Development Approach

6.1. Methodology:

- Agile development with iterative updates
 - Designed controllers tested with postman and integrated with frontend.
 - Tested frontend with JSON data before integrating with backend.
- CI/CD for continuous deployment.

6.2. Development Phases:

- Requirement Analysis & UI/UX Design (Figma for wireframes)
- Frontend Development
- Database Design
- Backend Design
- Database Integration & API Implementation
- Security Enhancements & Payment Gateway Integration
- Testing & Debugging
- Deployment & Maintenance

7. Risk Assessment & Mitigation

Identified Risks:

Payment Failures: Implement retry mechanisms.

OTP Delay Issues: Use Redis for faster processing.

Unauthorized Access: Implement strict JWT-based authentication.

8. Testing Strategy & Logging

Unit Testing: Use JUnit & Mockito for backend testing.

Integration Testing: Use Postman for API testing.

User Acceptance Testing: Test real mobile numbers with simulated payment methods(card payment , UPI payment and net banking)

9. Project Timeline & Milestones

Phases	Start Date	End Date	Task Done
Requirement Analysis and UI/UX Design	10 FEB 2025	13 FEB 2025	-Gathered and analysed scope and requirements for the project -design the user friendly interfaces using figma
Phase I – Frontend Development and DB Design	14 Feb 2025	22 Feb 2025	-Developed the frontend design using html , css , bootstrap and js -used json data to populate and test the frontend working -designed the database for the application
Phase II – Backend Development	12 March 2025	21 March 2025	-Developed the layered backend with spring boot (Maven based) -implemented various controllers to handle the http request and response -implemented role based authentication and authorization with spring security (JWT Token)

Useful Links

Project Source Code : https://github.com/Nanthene-Elango/Mobile_Prepaid_Recharge_Application.git

Project Demo: https://www.linkedin.com/posts/nanthene-r-e-38b656227_javafullstack-springboot-backenddevelopment-activity-7309447186038104064-ezRi?utm_source=share&utm_medium=member_desktop&rcm=ACoAADje95sBJk95ka97jQ4xhhqNLGigHDc3rQo

Project UI Design Update: https://www.linkedin.com/posts/nanthene-r-e-38b656227_javafullstack-webdevelopment-frontend-activity-7302134277758930944-Zwt6?utm_source=share&utm_medium=member_desktop&rcm=ACoAADje95sBJk95ka97jQ4xhhqNLGigHDc3rQo